

KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 3

Character LCD1602 Interfacing via PCF8574 I2C Backpack

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this laboratory task is to interface an alphanumeric HD44780-compatible 16×2 character LCD over the I^2C bus using a PCF8574 8-bit expander backpack. Students analyze 4-bit nibble packetization, evaluate address mismatch fault handling, implement dynamic horizontal text scrolling, and calculate wire-level transmission throughput and display refresh bandwidth.

1 Hardware Architecture & Backpack Mapping

The standard Hitachi HD44780 LCD controller requires a minimum of 6 control/data lines in 4-bit parallel mode ($RS, E, D_4, D_5, D_6, D_7$). The "1602 IIC" backpack employs a PCF8574 remote 8-bit I/O expander to bridge the two-wire I^2C bus to the parallel LCD interface pins.

Table 1: PCF8574 Output Pin to HD44780 LCD Signal Mapping

PCF8574 Port	Bit Mask	LCD Signal	Functional Description
P_0	`0x01`	**RS** (Register Select)	`0` = Command Register, `1` = Data (Character) Register
P_1	`0x02`	**RW** (Read/Write)	Held permanently at `0` (Write mode only)
P_2	`0x04`	**E** (Enable)	High-to-Low transition ($E \downarrow$) latches data on D_4–D_7
P_3	`0x08`	**Backlight**	Controls NPN transistor for LED backlight (`1` = ON, `0` = OFF)
P_4	`0x10`	**D4**	LCD Data Bit 4
P_5	`0x20`	**D5**	LCD Data Bit 5
P_6	`0x40`	**D6**	LCD Data Bit 6
P_7	`0x80`	**D7**	LCD Data Bit 7 (also Busy Flag BF in read mode)

THEORETICAL FOUNDATION & PROTOCOL RULES

Enable Latch Timing Physics: To transfer one byte to the LCD in 4-bit mode, the byte is partitioned into two 4-bit nibbles. Each nibble is sent twice: first with $E = 1$ (`0x04`), then with $E = 0$. The falling edge of E latches the nibble into the LCD. Because each I^2C byte transfer over a 100 kHz bus takes $90\,\mu$s, it inherently satisfies the HD44780 minimum enable pulse width ($t_{w(E)} \geq 450$ ns) without requiring microsecond delay instructions.

2 Firmware Implementation

2.1 Platform 1: STM32 HAL Low-Level Driver (main.c)

In the STM32 HAL implementation, we construct an atomic 4-byte buffer for every character: [High Nibble | E, High Nibble and transmit it in a single I^2C transaction.

Listing 1: STM32 HAL PCF8574 LCD Driver with Scrolling Support

```c
#define LCD_ADDR (0x27 << 1) // 7-bit 0x27 shifted left by 1
#define LCD_RS   0x01
#define LCD_EN   0x04
#define LCD_BL   0x08

static void lcd_nibble(uint8_t nib, uint8_t rs) {
    uint8_t v = (uint8_t)((nib << 4) | LCD_BL | (rs ? LCD_RS : 0));
    uint8_t d[2] = { (uint8_t)(v | LCD_EN), v };
    HAL_I2C_Master_Transmit(&hi2c1, LCD_ADDR, d, 2, 50);
}

static void lcd_byte(uint8_t b, uint8_t rs) {
    uint8_t hi = (uint8_t)((b & 0xF0) | LCD_BL | (rs ? LCD_RS : 0));
    uint8_t lo = (uint8_t)(((b << 4) & 0xF0) | LCD_BL | (rs ? LCD_RS : 0));
    uint8_t d[4] = { (uint8_t)(hi | LCD_EN), hi, (uint8_t)(lo | LCD_EN), lo };
    HAL_I2C_Master_Transmit(&hi2c1, LCD_ADDR, d, 4, 50);
}

void lcd_init(void) {
    HAL_Delay(50);
    lcd_nibble(0x03, 0); HAL_Delay(5);
    lcd_nibble(0x03, 0); HAL_Delay(1);
    lcd_nibble(0x03, 0); HAL_Delay(1);
    lcd_nibble(0x02, 0); HAL_Delay(1); // Force 4-bit mode
    lcd_byte(0x28, 0); // 4-bit mode, 2-line display, 5x8 font
    lcd_byte(0x0C, 0); // Display ON, Cursor OFF, Blink OFF
    lcd_byte(0x01, 0); HAL_Delay(2); // Clear display
    lcd_byte(0x06, 0); // Entry mode: increment cursor
}

void lcd_set_cursor(uint8_t row, uint8_t col) {
    lcd_byte((uint8_t)(0x80 | (row ? 0x40 : 0x00) | col), 0);
}

void lcd_print(const char *s) {
    while (*s) lcd_byte((uint8_t)*s++, 1);
}

/* Horizontal scrolling implementation for strings > 16 characters */
void lcd_scroll_message(uint8_t row, const char *msg, uint16_t delay_ms) {
    int len = strlen(msg);
    char window[17];
    for (int i = 0; i <= len - 16; i++) {
        strncpy(window, msg + i, 16);
        window[16] = '\0';
        lcd_set_cursor(row, 0);
        lcd_print(window);
        HAL_Delay(delay_ms);
    }
}
```

2.2 Platform 2: Arduino Uno Sketch (Wire + LiquidCrystal_I2C)

Listing 2: Arduino Uno LCD Program with Dynamic Scrolling

```arduino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const char line1_msg[] = "Hello, Uganda";
const char scroll_msg[] = "    Kyambogo University Dept of EEE - TETE 3102
    Microcontrollers    ";

void setup() {
  Serial.begin(115200);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print(line1_msg);
}

void loop() {
  int len = strlen(scroll_msg);
  char window[17];
  for (int i = 0; i <= len - 16; i++) {
    strncpy(window, scroll_msg + i, 16);
    window[16] = '\0';
    lcd.setCursor(0, 1);
    lcd.print(window);
    delay(300);
  }
}
```

3 Fault Injection: Address Mismatch Investigation

In this experiment, the code was deliberately reconfigured with an incorrect address (`0x28` instead of `0x27`).

Table 2: System Behavior Under Address Mismatch (`0x28`)

Platform	Function Call	Returned Status Code	Physical Observation & Bus State
STM32 HAL	`HAL_I2C_Master_Transmit`	`HAL_ERROR` (`0x01`) Error Flag: `HAL_I2C_ERROR_AF` (`0x04`)	The LCD remains completely blank or shows dark contrast blocks. No ACK is generated on the 9th clock cycle of the address byte. The transmission aborts immediately.
Arduino Wire	`Wire.endTransmission()`	Return code **2** (`NACK on address`)	Text fails to appear. The driver receives a NACK after sending the invalid address `0x28` and refuses to stream subsequent character bytes.

4 Timing & Wire Throughput Analysis (Task Question 4)

Task Question 4

The STM32 driver sends four bytes per LCD character in one transaction. How many bits are on the wire per character at 100 kHz and how long does it take? How many characters per second can the display be refreshed at most?

1. Bit Count on the Physical Wire per Character

In our driver, one LCD character is transmitted as a single atomic transaction consisting of:

- 1 START condition (S)
- 1 Slave Address Byte (`0x4E`): 8 data bits + 1 ACK bit **= 9 clock cycles**
- 4 Payload Bytes (Upper nibble with $E = 1$, Upper nibble with $E = 0$, Lower nibble with $E = 1$, Lower nibble with $E = 0$):

$$4 \text{ data bytes} \times (8 \text{ data bits} + 1 \text{ ACK bit}) = 4 \times 9 = \mathbf{36} \text{ clock cycles} \tag{1}$$

- 1 STOP condition (P)

Total clock periods on the wire per character:

$$N_{\text{clocks}} = 9 \text{ (address)} + 36 \text{ (data)} = \mathbf{45} \text{ SCL clock periods} \tag{2}$$

2. Transmission Duration at 100 kHz ($T_{\text{SCL}} = 10.0\,\mu\text{s}$)

$$t_{\text{char}} = 45 \text{ clocks} \times 10.0\,\mu\text{s/clock} = \mathbf{450}\,\mu\text{s} \quad (0.45 \text{ ms}) \tag{3}$$

(If calculating pure data payload without address re-transmission in a streaming transaction: $36 \times 10\,\mu\text{s} = 360\,\mu\text{s}$).

3. Maximum Characters Refreshed Per Second

The theoretical maximum continuous character transfer rate across the bus:

$$\text{Throughput} = \frac{1}{t_{\text{char}}} = \frac{1}{450 \times 10^{-6} \text{ s}} \approx \mathbf{2222.2} \text{ characters/second} \tag{4}$$

Full Screen Refresh Rate: A 16×2 LCD has 32 visible character cells. Refreshing all 32 cells requires:

$$t_{\text{screen}} = 32 \times 450\,\mu\text{s} = 14.4 \text{ ms} \implies \text{Max Frame Rate} = \frac{1}{0.0144 \text{ s}} \approx \mathbf{69.4} \text{ frames/second} \tag{5}$$

This demonstrates that even at standard 100 kHz speeds, the I^2C bus can refresh the entire display at ≈ 70 Hz, far surpassing human visual persistence.